

Laboratorio 3: Tiny MC

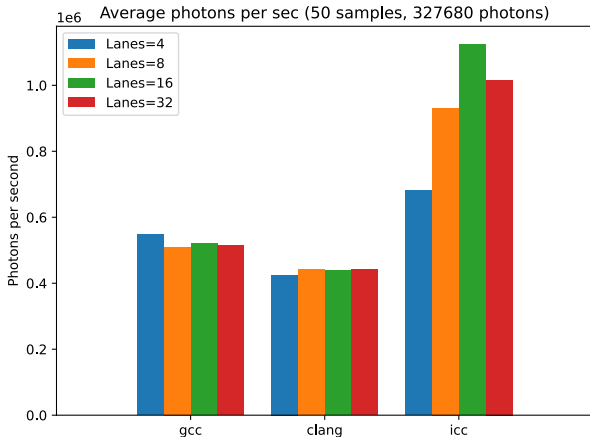
Computación Paralela - FAMAF

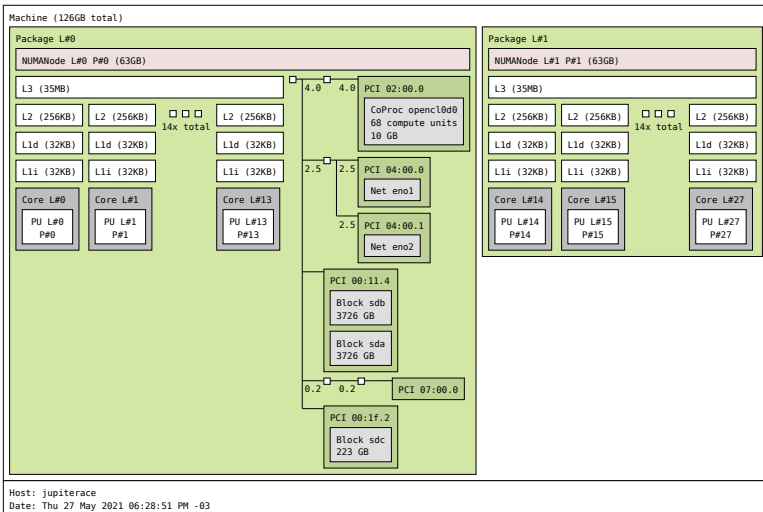
Javier Cremona
Marcelo Castillo

May 30, 2021

Resultados Laboratorio 2 en JupiterAce

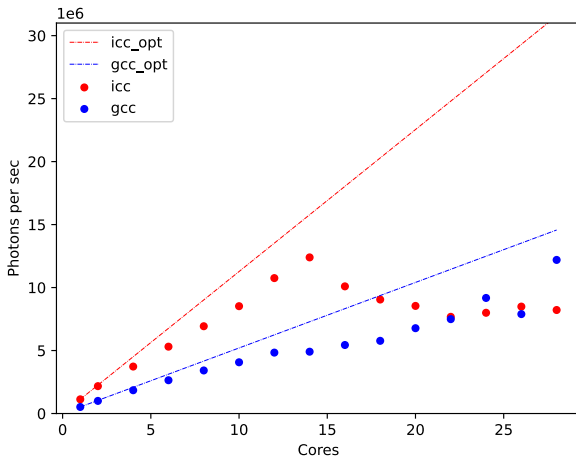
Mejor resultado: 1.1 M photons per second (icc, 327680 fotones simulados y 16 lanes)





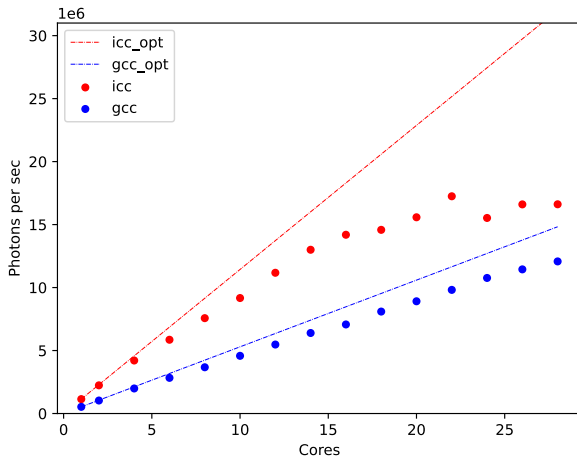
Cores vs pps

Average photons per sec (25 samples, 327680 photons, 16 lanes)



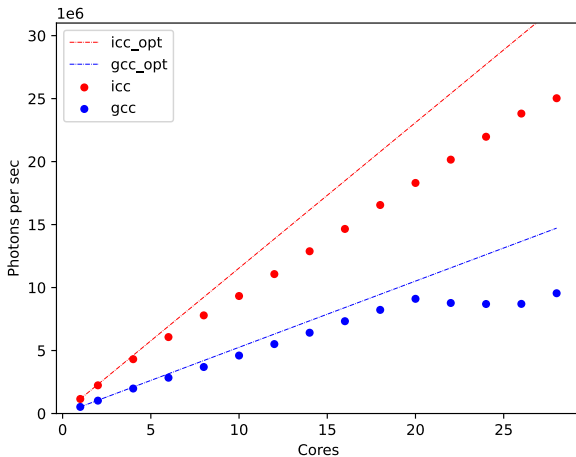
Cores vs pps (x10 la cantidad de fotones)

Average photons per sec (25 samples, 3276800 photons, 16 lanes)



Cores vs pps ($\times 100$ la cantidad de fotones)

Average photons per sec (25 samples, 32768000 photons, 16 lanes)



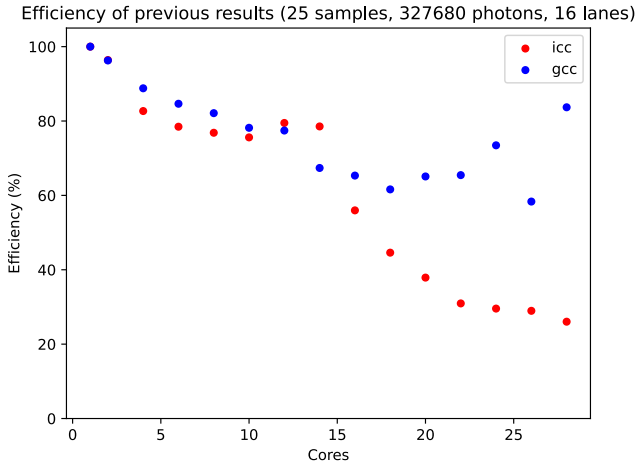
Calculando la eficiencia

Modificamos el cálculo de la eficiencia que vimos en teoría. En lugar de usar tiempo, usamos photons per second (pps).

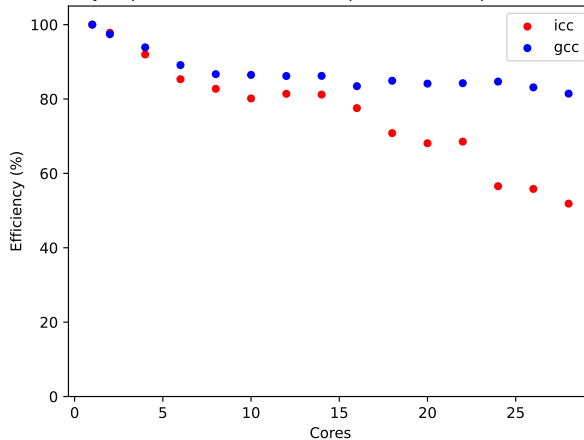
$$E_n = \frac{P_n}{P_1 * n}$$

donde

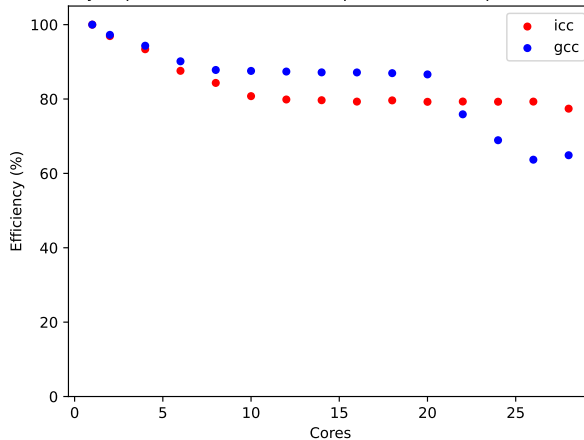
- n es la cantidad de cores utilizados,
- P_1 es la cantidad de pps obtenida con la versión secuencial y
- P_n es la cantidad de pps obtenida con la versión paralelizada que utiliza n cores.



Efficiency of previous results (25 samples, 3276800 photons, 16 lanes)



Efficiency of previous results (25 samples, 32768000 photons, 16 lanes)



- La primera ejecución que recolecta las métricas al usar FDO es mucho más lenta al compilar con `-fopenmp`. Para `icc`, con el máximo valor de fotones empleado, esa primera ejecución llegó a ser más de 100 veces más lenta que el tiempo que demora una ejecución del binario optimizado.
- Esto nos impidió probar con cantidades más grandes de fotones simulados (~ 10 minutos corriendo solamente la ejecución de FDO).
- Para esta ejecución que demora mucho, `numatop` reportó valores de ratio RMA/LMA mayores a 1.
- No pudimos descifrar la salida de `perf c2c`.

- Al usar OpenMP, logramos que el código corra 22 veces más rápido.
- La eficiencia cae al usar los 28 cores.
- Tuvimos que aumentar la cantidad de fotones simulados para que valga la pena usar la máxima cantidad de cores disponibles.